

基于华为云与 ESP32 的智能垃圾分类系统设计实验报告

陈鑫 李彦萱

April 2026

1 摘要

本项目设计并实现了一个基于 ESP32-S3-CAM 终端、华为云服务及通义千问视觉大模型的端到端智能垃圾分类 AIoT 系统。我们采用了多端协同的分层架构。在硬件端，我们以 ESP32-S3 为核心，利用有限状态机驱动 OV3660 摄像头进行图像采集，并通过 AWS Signature V4 签名算法将图像安全上传至华为云 OBS 存储。通过华为云 IoTDA 实现设备管理与命令下发。并且我们创新性地采用了 OBS 预签名 URL 方案调用视觉大模型，有效解决了嵌入式设备在处理大体积 Base64 数据时的内存与带宽限制。同时我们开发了集成华为云 IAM 认证与 OBS 签名算法的微信小程序，实现了一键“拍照识别”、实时结果轮询以及识别历史记录的可视化展示。通过样例测试，我们发现该系统端到端响应延迟约 5-10 秒，能够准确实现可回收垃圾与其它垃圾的自动化识别与物理分类执行。

2 设计目的

本课程设计的核心目标是构建一个端到端的智能物联网系统，通过集成嵌入式硬件、华为云平台及大模型 AI 能力，解决传统垃圾分类中效率低下及人工识别成本高的问题。具体设计目的包括：

1. **实现垃圾分类的自动化与智能化**：利用先进的视觉大模型替代人工识别，实现对常见生活垃圾的精准判别与自动物理归类，探索利用 AIoT 技术推动“无废城市”建设的有效路径。
2. **实践多层 AIoT 架构**：深入探索并实现从底层硬件采集、网络传输管理、云端存储与计算到移动端应用的全链路协同，理解物联网系统的整体运作机制。
3. **探索受限资源下的高性能方案**：针对 ESP32-S3 等嵌入式设备内存资源受限的特点，通过引入 OBS 预签名 URL 等技术手段，优化图像识图链路，探索降低设备负载并提升系统响应速度的最优解。
4. **构建闭环的远程监控与管理系统**：通过微信小程序实现远程指令触发、实时状态反馈及识别历史数据的持久化管理，提升系统的实用性与用户交互体验。

3 实验原理

我们的系统的核心运行逻辑建立在硬件端的状态调度与云端的数据流转机制之上。

3.1 硬件层：基于有限状态机的逻辑调度

系统硬件端采用有限状态机作为核心调度框架。通过将业务逻辑解耦为独立的状态(如 IDLE、CAPTURE、UPLOAD、RECOGNIZE 等)，系统能够确保在嵌入式环境下实现高效的流程控制与错误隔离。每个状态通过确定的触发条件进行转移，这种非阻塞设计避免了单点故障导致的系统死锁。硬件构成原理图如下：

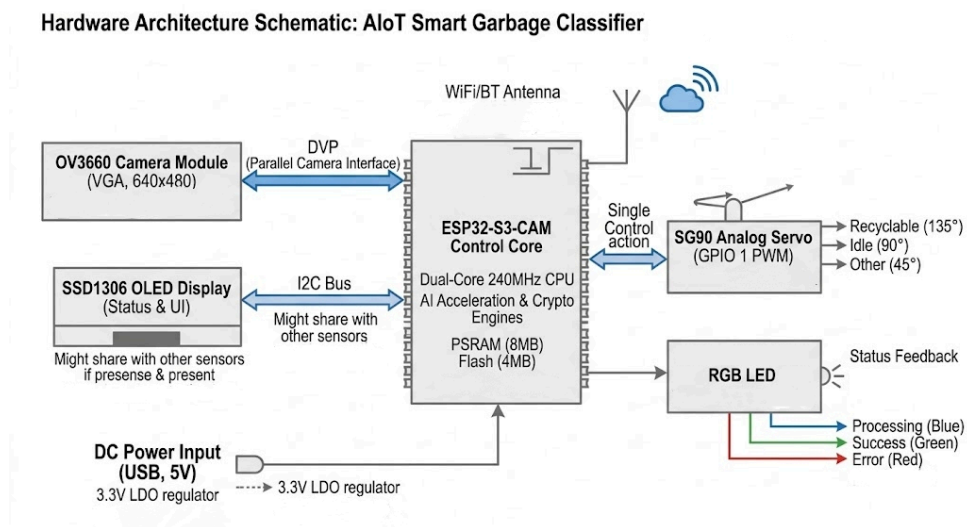


图 1: 硬件构成原理图

3.2 平台接入层：物模型与数字孪生通信

我们利用华为云 IoTDA 平台实现物理设备向数字孪生的映射。

- **MQTT 通信协议**：设备通过 MQTT 协议与云端建立双向连接，从而支持长连接下的低时延指令下发与属性上报。
- **物模型定义**：通过定义 `GarbageClassification` 服务，系统将垃圾种类、识别置信度及 OBS 图片路径等物理参数抽象为结构化的属性列表。这种物模型化设计使得应用端无需关心底层硬件细节即可通过标准 API 获取设备状态。

3.3 云端计算层：规则引擎与自动化运维

云端通过华为云规则引擎实现数据流的自动化路由。

- **报文过滤与提取**：系统配置 SQL 语句实时拦截设备上报的消息，并利用条件过滤机制精准捕获识别结果。
- **异步数据持久化**：规则引擎将匹配的数据自动转发至 OBS 桶中，并以追加写入的方式更新 `records.csv` 文件。这种无服务器化的持久化方案，既保证了数据的实时记录，又极大地简化了系统后端架构。

3.4 智能识别层：低负载识图与安全认证

针对 ESP32-S3 等嵌入式设备处理大体积数据时的资源限制，我们引入了先存后识的原理：

- **安全认证机制**：利用 AWS Signature V4 算法在设备端生成二进制 HMAC 签名，确保图片在传输过程中不可被篡改且仅限授权上传。
- **预签名 URL 技术**：设备上传图片后，生成一个带有临时访问权限的预签名 URL 发送给 DashScope AI 接口。AI 服务直接通过该 URL 拉取图片进行深度学习推理，从而规避了板端发送大体积 Base64 报文带来的内存溢出风险。

3.5 微信小程序层：异步状态同步与可视化原理

小程序端通过轮询机制与云端实现异步状态同步。

- **数据解析逻辑**：通过 OBS SDK 实时获取并解析 records.csv 文件中的历史数据，通过对比记录条数的变化来感知硬件识别任务的完成情况。
- **交互控制原理**：利用华为云 IAM Token 认证调用控制 API，将用户的点击行为转化为标准的云端控制命令，实现对物理终端的越程操作。

4 系统设计与实现

我们通过模块化分层架构与有限状态机调度，确保了系统在资源受限环境下的稳定性与实时性。

4.1 系统总体架构概览

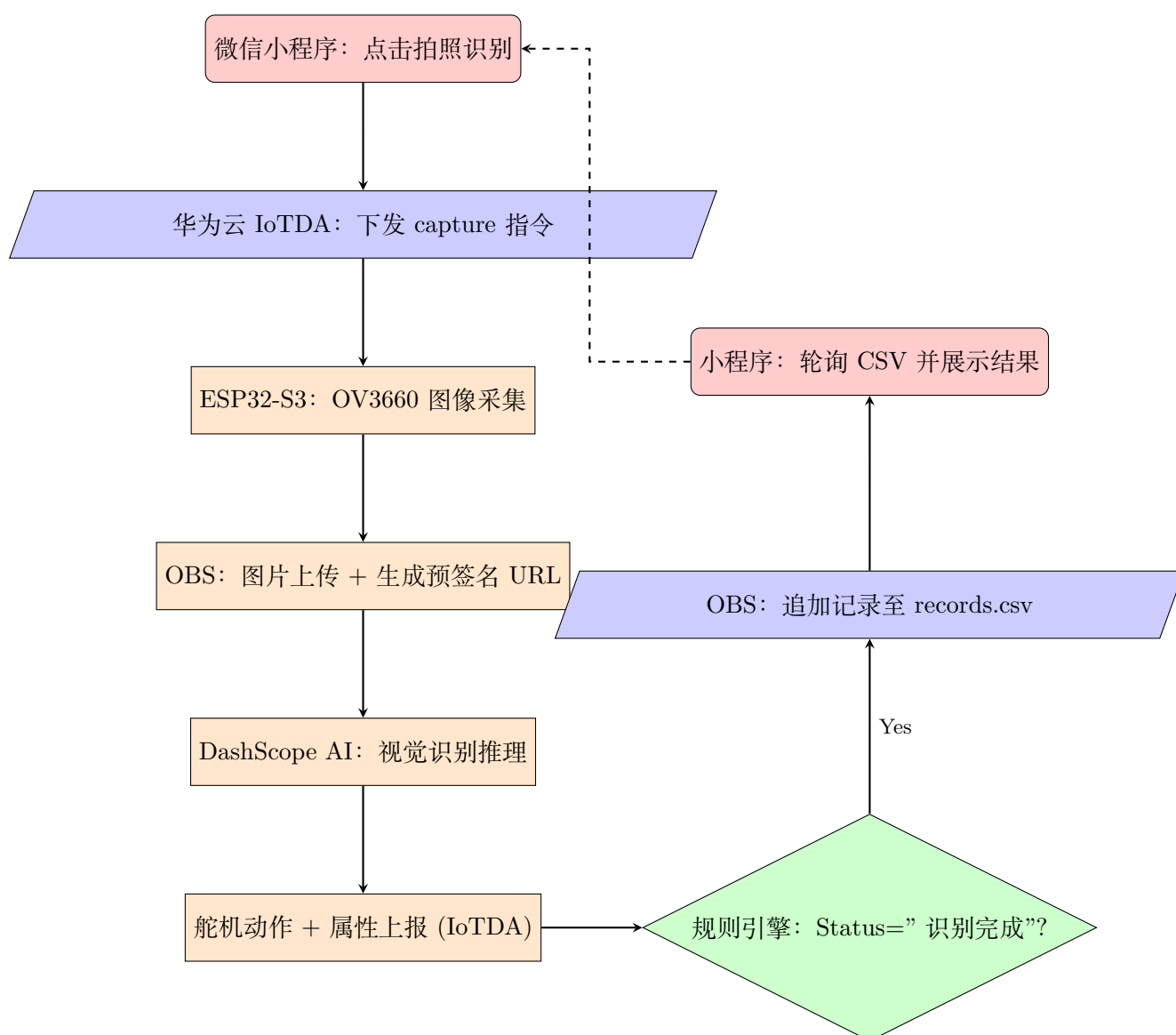


图 2: 智能垃圾分类系统逻辑流程图

我们的系统采用四层架构设计，实现了从物理感知到应用表现的全链路闭环：

- **业务协调层**：位于最顶层，由状态机主逻辑组成，负责调度各功能模块并处理异常。
- **功能模块层**：包含网络维护、图像采集、舵机控制及显示驱动等独立模块。

- **云服务层**：集成华为云 IoTDA、OBS 与 DashScope 服务。
- **硬件层**：直接驱动 ESP32-S3、OV3660 摄像头、SSD1306 OLED 及外置 LED 等物理硬件。

4.2 数据流逻辑与全链路闭环

系统核心数据流经过六个关键阶段，形成完整的端到端链路：

1. **触发阶段**：微信小程序通过华为云 IoTDA API 下发 `capture` 异步指令。
2. **采集阶段**：ESP32 硬件端捕获指令后，驱动 OV3660 拍摄并将 JPEG 数据存入 PSRAM 帧缓冲区。
3. **存储阶段**：利用 AWS SigV4 算法将图片上传至 OBS 桶，并生成具有时效性的预签名 URL。
4. **推理阶段**：AI 接口通过预签名 URL 抓取图片，进行逻辑推理并返回包含类别与置信度的 JSON 报文。
5. **反馈阶段**：硬件端驱动舵机执行分类动作，并同步上报属性至 IoTDA 物模型。
6. **结果呈现**：云端规则引擎将识别结果自动存入 `records.csv`，小程序通过轮询该文件实现前端结果更新。

4.3 设备端实现

硬件层作为系统的物理载体，主要负责图像采集、实时状态反馈以及物理执行动作，包含摄像头、LED 灯、舵机等硬件模块。在设计硬件层时我们利用了 ESP32-S3 的双核处理能力与硬件加速特性。

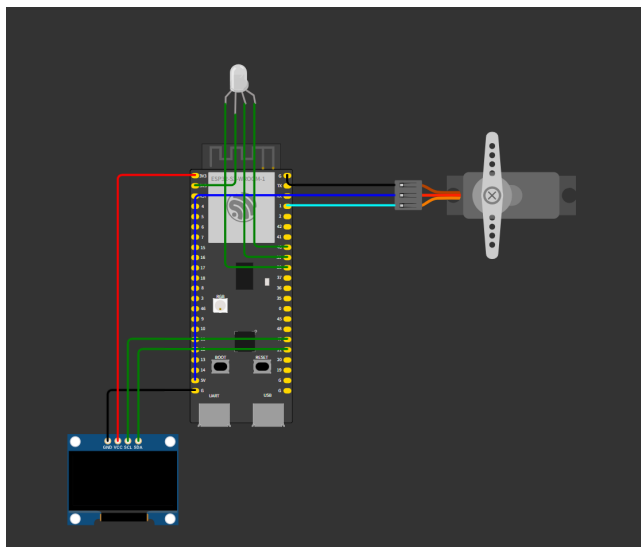


图 3: 硬件连接

A. 控制核心与存储管理：系统核心选用 ESP32-S3-CAM 微控制器。针对图像处理的高内存需求，系统在代码实现中引入了以下优化：

- **PSRAM 自动检测机制**：在 `camera.cpp` 中，系统通过 `psramFound()` 动态检测外部伪静态随机存储器。若检测成功，则将帧缓冲区（Frame Buffer）置于 PSRAM 中，并将 `jpeg_quality` 提升至 10，以支持更高质量的图像采集。
- **多重初始化序列**：为保证系统稳定性，硬件初始化遵循严格的逻辑顺序。系统通过 `main.ino` 的 `setup()` 函数依次启动 WDT、OLED、WiFi 及摄像头模块。任何关键模块（如摄像头、舵机）初始化失败，系统都会捕获 `ErrorCode` 并通过 LED 红灯示警。

Listing 1: MQTT 下行指令解析与状态触发逻辑

```

1 void mqttCallback(char* topic, byte* payload, unsigned int length) {
2     INFO_PRINTLN("\n收到MQTT消息:");
3     INFO_PRINTF("主题: %s\n", topic);
4     // 安全处理: 检查消息长度 (此处省略)
5     // 复制消息到缓冲区
6     char payloadBuffer[MAX_PAYLOAD_SIZE];
7     memcpy(payloadBuffer, payload, length);
8     payloadBuffer[length] = '\0';
9     INFO_PRINTF("内容: %s\n", payloadBuffer);
10
11     // 优先提取request_id
12     char requestId[64] = "";
13     const char* idStart = strstr(topic, "request_id=");
14     if (idStart != NULL) {
15         idStart += 11; // 跳过 "request_id="
16         strncpy(requestId, idStart, sizeof(requestId) - 1);
17         requestId[sizeof(requestId) - 1] = '\0';
18         INFO_PRINTF("Request ID: %s\n", requestId);
19     } else {
20         WARN_PRINTLN("无法提取request_id, 跳过命令处理");
21         return; // 没有request_id就无法响应, 只能放弃
22     }
23     // 解析JSON
24     JsonDocument doc;
25     DeserializationError error = deserializeJson(doc, payloadBuffer);
26     if (error) {
27         ERROR_PRINTLN("JSON解析失败");
28         ERROR_PRINTF("错误: %s\n", error.c_str());
29         // JSON解析失败也发送响应 (带错误码), 避免IoTDA等待超时
30         sendCommandResponse(requestId);
31         return;
32     }
33     // 提取命令参数
34     const char* command = doc["command_name"] | "";
35     INFO_PRINTF("命令: %s\n", command);
36     if (strcmp(command, "capture") == 0) { // 触发拍照识别
37         INFO_PRINTLN("收到拍照命令");
38         captureTriggered = true; // 触发状态机进入CAPTURE状态
39
40     } else if (strcmp(command, "test") == 0) { // 测试命令
41         INFO_PRINTLN("收到测试命令");
42         displayMessage("测试", "命令接收", "成功");
43     } else { // 未知命令也发送响应, 不让IoTDA挂起
44         WARN_PRINTF("未知命令: %s\n", command);
45     }
46     // 发送命令响应
47     sendCommandResponse(requestId);
48 }

```

B. 视觉采集与图像调优：系统采用 OV3660 图像传感器，通过 DVP 接口与 MCU 连接。

- **传感器配置：**采集分辨率设定为 VGA (640x480)，像素格式为 JPEG。为适配垃圾桶垂直安装的角度，代码中显式调用 `s->set_vflip(s, 1)` 进行垂直翻转调优。
- **双缓冲区方案：**在 PSRAM 环境下启用双帧缓冲区，有效缓解了图像传输与数据处理之间的吞吐量矛盾，提升了连拍稳定性。

Listing 2: OV3660 传感器配置与图像调优

```
1 bool initCamera() {
2     INFO_PRINTLN("初始化摄像头...");
3     // 摄像头配置结构体
4     camera_config_t config;
5     (省略引脚定义....)
6     config.xclk_freq_hz = 20000000; // 20MHz XCLK
7     config.frame_size = CAMERA_FRAMESIZE; // 分辨率
8     config.pixel_format = CAMERA_PIXFORMAT; // 输出格式
9     // 具体设置在config.h中
10    config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
11    config.fb_location = CAMERA_FB_IN_PSRAM; // 使用PSRAM存储帧缓冲区
12    config.jpeg_quality = CAMERA_QUALITY;
13    config.fb_count = CAMERA_FB_COUNT;
14    // 如果检测到PSRAM，使用更高质量的配置
15    if(psramFound()){
16        config.jpeg_quality = 10; // 进一步提升JPEG质量
17        config.fb_count = 2; // 双缓冲区，提升稳定性
18        config.grab_mode = CAMERA_GRAB_LATEST;
19        INFO_PRINTLN("检测到PSRAM，使用高质量配置");
20    } else { // 如果没有PSRAM，限制帧大小并使用DRAM
21        config.frame_size = FRAMESIZE_SVGA;
22        config.fb_location = CAMERA_FB_IN_DRAM;
23        WARN_PRINTLN("未检测到PSRAM，使用SVGA分辨率");
24    }
25    // 初始化摄像头
26    esp_err_t err = esp_camera_init(&config);
27    if (err != ESP_OK) {
28        ERROR_PRINTF("摄像头初始化失败: 0x%x\n", err);
29        return false;
30    }
31    // 获取传感器对象并配置
32    sensor_t *s = esp_camera_sensor_get();
33    if (s == nullptr) {
34        ERROR_PRINTLN("无法获取传感器对象");
35        return false;
36    }
37    // 配置传感器参数（优化图像质量）（具体配置此处省略）
38    INFO_PRINTLN("摄像头初始化成功");
39    printCameraInfo();
40
41    return true;
42 }
```

C. 机械执行机构设计：物理分类通过单舵机系统实现，采用角度脉宽调制控制。在模型中我们为了减少复杂度未实装垃圾桶，采用舵机转动来表示垃圾桶的开关，同时因为同时接入两个舵机的电流过大，在模型中我们暂且采用单个舵机的不同方向转动来表现两种不同的垃圾。

- **角度映射逻辑：**舵机连接于 GPIO 1。根据 servo.cpp 中的实现，我们将 90° 设定为复位中位，45° 定义为“其它垃圾”投放位置，135° 定义为“可回收垃圾”投放位置。
- **动作时序控制：**执行逻辑包含开启延迟与保持时间，确保投放物有足够时间滑入分类仓，随后舵机自动复位至中位。

Listing 3: 基于 PWM 的舵机分类控制逻辑实现

```
1 void openBin(GarbageType type) {
2     INFO_PRINTF("打开垃圾桶: %s\n", GARBAGE_NAMES[type]);
3     // 显示状态
4     displayMessage("Processing", GARBAGE_NAMES[type], "Opening...");
5     // 设置LED为处理中（蓝色）
6     setLEDColour(LED_BLUE);
7     if (type == GARBAGE_RECYCLABLE) {
8         // 可回收垃圾：向右摆动
9         INFO_PRINTLN("可回收垃圾 - 舵机向右摆动");
10        servo.write(SERVO_ANGLE_RIGHT); // 135°
11        delay(500); // 打开动作时间
12        // 保持开启状态
13        delay(SERVO_HOLD_TIME);
14        // 回位
15        servo.write(SERVO_ANGLE_CENTER); // 90°
16        delay(500); // 回位动作时间
17
18    } else if (type == GARBAGE_OTHER) {
19        // (逻辑与可回收相同，故此处省略)
20    }
21    // 设置LED为成功（绿色）
22    setLEDColour(LED_GREEN);
23    displayMessage("Processing Complete", GARBAGE_NAMES[type], "Categorized");
24    INFO_PRINTLN("垃圾桶动作完成");
25 }
```

D. 本地 HMI 与状态交互：为提升用户体验与可维护性，我们集成了多维度反馈机制。

- **状态指示灯控制：**通过 FastLED 库驱动外置 LED 灯。我们定义了颜色状态机：蓝色代表“启动或处理中”，绿色代表“空闲或成功”，红色代表“系统故障”。
- **多级 OLED 显示：**基于 SSD1306 驱动实现三行状态同步展示。除常规识别结果外，系统还实现了进度条显示及网络状态栏实时显示 WiFi 与 MQTT 连接状态。

Listing 4: 本地 HMI 状态显示与 LED 反馈实现

```
1 // display.cpp
2 void displayMessage(const char* line1, const char* line2, const char* line3) {
3     display.clearDisplay();
4     // 设置字体大小
5     display.setTextSize(1);
```

```

6     display.setTextColor(SSD1306_WHITE);
7     // 显示第一行
8     display.setCursor(0, 0);
9     display.println(line1);
10    // 显示第二行
11    display.setCursor(0, 16);
12    display.println(line2);
13    // 显示第三行
14    display.setCursor(0, 32);
15    display.println(line3);
16    display.display();
17
18    DEBUG_PRINTF("[OLED] %s | %s | %s\n", line1, line2, line3);
19 }
20 // led.cpp
21 void setLEDColor(LEDColor color) {
22     if (USE_ONBOARD_RGB_LED) {
23         // LED
24         switch (color) {
25             (初始判定省略)
26         }
27         FastLED.show();
28     } else { // 外部LED
29         digitalWrite(LED_RED_PIN, LED_STATE_OFF);
30         digitalWrite(LED_GREEN_PIN, LED_STATE_OFF);
31         digitalWrite(LED_BLUE_PIN, LED_STATE_OFF);
32         switch (color) {
33             case LED_OFF: // 全部关闭
34                 break;
35             case LED_RED:
36                 digitalWrite(LED_RED_PIN, LED_STATE_ON);
37                 break;
38             case LED_GREEN:
39                 digitalWrite(LED_GREEN_PIN, LED_STATE_ON);
40                 break;
41             case LED_BLUE:
42                 digitalWrite(LED_BLUE_PIN, LED_STATE_ON);
43                 break;
44             case LED_YELLOW:
45                 digitalWrite(LED_RED_PIN, LED_STATE_ON);
46                 digitalWrite(LED_GREEN_PIN, LED_STATE_ON);
47                 break;
48             case LED_PURPLE:
49                 digitalWrite(LED_RED_PIN, LED_STATE_ON);
50                 digitalWrite(LED_BLUE_PIN, LED_STATE_ON);
51                 break;
52             case LED_WHITE:
53                 digitalWrite(LED_RED_PIN, LED_STATE_ON);
54                 digitalWrite(LED_GREEN_PIN, LED_STATE_ON);
55                 digitalWrite(LED_BLUE_PIN, LED_STATE_ON);
56                 break;
57         }

```

```

58     }
59     DEBUG_PRINTF("[LED] 设置颜色: %d\n", color);
60 }

```

E. 有限状态机实现：系统的软件核心是运行于 `main.ino` 中的有限状态机 (FSM)。该架构通过以下机制确保了复杂业务的高效调度：

- **状态流转控制：**系统通过全局变量 `captureTriggered` 异步监听来自 `iotda.cpp` 的 MQTT 指令。一旦触发，系统由 `STATE_IDLE` 切换至 `STATE_CAPTURE` 进行图像采集，获取图像指针后流转至 `STATE_UPLOAD` 执行云端同步。
- **异常隔离机制：**各状态处理函数均具备独立的错误检测逻辑。若图像采集、OBS 上传或 AI 识别中任何环节出现故障，状态机将立即捕获异常并切换至 `STATE_ERROR` 状态。在错误状态下，系统通过 LED 红灯进行物理警示，并在 `delay(5000)` 后自动恢复至 `STATE_IDLE` 状态，有效避免了因单点硬件故障导致的系统永久性死锁。

Listing 5: `main.ino` 中的核心状态机调度逻辑

```

1  void loop() {
2      // 1. 防止系统死机
3      esp_task_wdt_reset();
4      // 2. 网络维护层
5      checkAndReconnectWiFi();    // WiFi 自动重连
6      handleMQTTLoop();           // MQTT 消息处理
7      // 3. 业务逻辑层
8      stateMachineLoop();         // 状态机执行
9      checkAndReportStatus();     // 定时状态上报
10     // 4. 延时 (降低CPU占用)
11     delay(100);
12 }
13 // ===== 状态机循环 =====
14 void stateMachineLoop() {
15     switch (currentState) {
16         case STATE_IDLE:
17             // 空闲状态, 等待触发
18             if (captureTriggered) {
19                 INFO_PRINTLN("触发拍照流程");
20                 captureTriggered = false;
21                 currentState = STATE_CAPTURE;
22                 stateStartTime = millis();
23                 setLEDColor(LED_BLUE);
24                 displayMessage("Capturing...", "Please Wait", "");
25             }
26             break;
27         case STATE_CAPTURE: {
28             // 拍照状态
29             INFO_PRINTLN("开始拍照...");
30             photoBuffer = capturePhoto();
31
32             if (photoBuffer == NULL) {
33                 ERROR_PRINTLN("拍照失败");
34                 lastError = ERROR_CAMERA_CAPTURE;

```

```

35         currentState = STATE_ERROR;
36     } else {
37         INFO_PRINTF("拍照成功, 大小: %d 字节\n", photoBuffer->len);
38         currentState = STATE_UPLOAD; // 先上传并生成URL, 再识别
39         stateStartTime = millis();
40         displayMessage("Uploading...", "Please Wait", "");
41     }
42     break;
43 }
44 case STATE_UPLOAD: {
45     // 上传状态
46     INFO_PRINTLN("开始上传图片到OBS...");
47     // 生成时间戳
48     snprintf(timestamp, sizeof(timestamp), "%lu", millis());
49     if (uploadToOBS(photoBuffer->buf, photoBuffer->len, objectName, objectUrl)) {
50         INFO_PRINTLN("图片上传成功");
51         INFO_PRINTF("对象URL: %s\n", objectUrl);
52         if (!generatePresignedGetUrl(objectName, presignedImageUrl, sizeof(
53             presignedImageUrl), OBS_PREIGNED_URL_EXPIRES)) {
54             ERROR_PRINTLN("预签名URL生成失败");
55             lastError = ERROR_OBS_UPLOAD;
56             releasePhoto(photoBuffer);
57             photoBuffer = NULL;
58             currentState = STATE_ERROR;
59             break;
60         }
61         INFO_PRINTLN("预签名URL生成成功");
62         // 释放照片缓冲区 (上传完成后不再需要)
63         releasePhoto(photoBuffer);
64         photoBuffer = NULL;
65         currentState = STATE_RECOGNIZE;
66         stateStartTime = millis();
67         displayMessage("Recognizing...", "Please Wait", "");
68     } else {
69         ERROR_PRINTLN("图片上传失败");
70         lastError = ERROR_OBS_UPLOAD;
71
72         // 释放照片缓冲区
73         releasePhoto(photoBuffer);
74         photoBuffer = NULL;
75         currentState = STATE_ERROR;
76     }
77     break;
78 }
79 case STATE_RECOGNIZE: {
80     // 识别状态
81     INFO_PRINTLN("开始基于OBS URL的AI识别...");
82     if (recognizeGarbageByUrl(presignedImageUrl, garbageType, confidence)) {
83         INFO_PRINTF("识别成功: %s (%.2f%%)\n", GARBAGE_NAMES[garbageType],
84             confidence * 100);
85         // 上报识别结果 (包含稳定对象URL, 供后续系统使用)

```

```

85         reportResult(garbageType, confidence, objectName, objectUrl, timestamp);
86         currentState = STATE_EXECUTE;
87         stateStartTime = millis();
88         displayMessage("Result:", GARBAGE_NAMES[garbageType], "");
89     } else {
90         ERROR_PRINTLN("AI 识别失败");
91         lastError = ERROR_AI_RESPONSE;
92         currentState = STATE_ERROR;
93     }
94     break;
95 }
96 case STATE_EXECUTE: {
97     // 执行状态
98     INFO_PRINTLN("执行舵机动作...");
99     // 根据垃圾类型打开对应垃圾桶
100    openBin(garbageType);
101    // 上报状态
102    reportStatus("完成");
103    // 返回空闲状态
104    setLEDColour(LED_GREEN);
105    displayMessage("System Ready", "Awaiting Trigger", "");
106    currentState = STATE_IDLE;
107    INFO_PRINTLN("执行完成, 返回空闲状态");
108    break;
109 }
110 case STATE_ERROR:
111     // 错误状态
112     ERROR_PRINTF("系统错误: %s\n", ERROR_MESSAGES[lastError]);
113     setLEDColour(LED_RED);
114     displayError(ERROR_MESSAGES[lastError]);
115     // 上报错误
116     reportError(lastError);
117     // 清理资源
118     if (photoBuffer != NULL) {
119         releasePhoto(photoBuffer);
120         photoBuffer = NULL;
121     }
122     delay(5000);
123     currentState = STATE_IDLE;
124     lastError = ERROR_NONE;
125     setLEDColour(LED_GREEN);
126     displayMessage("System Ready", "Awaiting Trigger", "");
127     break;
128 }
129 }

```

4.4 平台接入层设计与实现

我们通过华为云 IoTDA 平台定义物模型实现了硬件端、云端与应用端之间标准化的数据交互与指令控制。

A. 物模型与属性定义：我们在构建系统时定义 GarbageClassification 物模型，将物理实体的状

态抽象为一组结构化的数据集，定义了以下属性

- **Status (String)**: 反映设备的实时业务阶，用于触发云端规则引擎。
- **GarbageType (String)**: 存储 AI 识别返回的类别结果。
- **Confidence (Decimal)**: 记录识别的置信度，为后续数据分析提供可靠性参考。
- **ImageUrl & ObjectName (String)**: 保存图片在 OBS 中的访问地址与对象索引，实现了媒体流与控制流的分离存储。
- **Timestamp (String)**: 同步识别发生的时间戳，确保历史记录时效性。

属性名称	数据类型	是否必填	默认值	备注
Status	string(字符串)	否	设备待机	设备待机
GarbageType	string(字符串)	否	垃圾	垃圾
Confidence	decimal(小数)	否	0.95	AI识别置信度
ImageUrl	string(字符串)	否	https://iotda-obs-data...	图片在OBS中的访问地址
ObjectName	string(字符串)	否	image/garbage_3546...	图片在OBS中的对象索引
Timestamp	string(字符串)	否	2020-04-26 12:27:21 GMT+08:00	识别时间戳

图 4: GarbageClassification 模型

属性名称	数据类型	是否必填	默认值	备注
Status	string(字符串)	否	设备待机	设备待机
GarbageType	string(字符串)	否	垃圾	垃圾
Confidence	decimal(小数)	否	0.95	AI识别置信度
ImageUrl	string(字符串)	否	https://iotda-obs-data...	图片在OBS中的访问地址
ObjectName	string(字符串)	否	image/garbage_3546...	图片在OBS中的对象索引
Timestamp	string(字符串)	否	2020-04-26 12:27:21 GMT+08:00	识别时间戳

图 5: GarbageClassification 属性

B. 异步命令下发与响应机制: 同时我们在物模型中定义了名为 `capture` 的控制命令，用于触发识别流程。

- **指令流转过程:** 微信小程序通过调用 IoTDA 的命令下发 API 触发业务。硬件端在回调函数中解析 JSON 报文，并提取 `request_id` 以进行同步响应，避免了云端接口超时。
- **架构解耦:** 通过全局变量 `captureTriggered` 驱动设备端状态机跳转，实现了 MQTT 通信模块与业务执行模块的逻辑解耦。

Listing 6: 基于 IoTDA 的 MQTT 指令解析与同步响应实现

```
1 void mqttCallback(char* topic, byte* payload, unsigned int length) {
2     INFO_PRINTLN("\n收到 MQTT 消息:");
3     INFO_PRINTF("主题: %s\n", topic);
4     // 安全处理: 检查消息长度 (省略...)
```

```

5 // 复制消息到缓冲区
6 char payloadBuffer[MAX_PAYLOAD_SIZE];
7 memcpy(payloadBuffer, payload, length);
8 payloadBuffer[length] = '\0';
9 INFO_PRINTF("内容: %s\n", payloadBuffer);
10 // 优先提取request_id (和之前的一样, 故省略)
11 // 解析JSON (和之前的一样, 故省略)
12 // 提取命令参数 (和之前的一样, 故省略)
13 // 处理不同命令
14 if (strcmp(command, "capture") == 0) {
15     // 触发拍照识别
16     INFO_PRINTLN("收到拍照命令");
17     captureTriggered = true; // 触发状态机进入CAPTURE状态
18 } else if (strcmp(command, "test") == 0) {
19     // 测试命令
20     INFO_PRINTLN("收到测试命令");
21     displayMessage("测试", "命令接收", "成功");
22 } else {
23     // 未知命令也发送响应, 不让IoTDA挂起
24     WARN_PRINTF("未知命令: %s\n", command);
25 }
26 // 发送命令响应
27 sendCommandResponse(requestId);
28 }
29 // ===== 发送命令响应 =====
30 void sendCommandResponse(const char* requestId) {
31     // 构建响应主题
32     char responseTopic[256];
33     snprintf(responseTopic, sizeof(responseTopic), "%s%s", MQTT_TOPIC_CMD_RESPONSE,
34             requestId);
35     // 发布响应 (使用预定义的响应数据) (省略)
36 }
37 void reportResult(GarbageType type, float confidence, const char* objectName, const char*
38     objectUrl, const char* timestamp) {
39     if (!mqttClient.connected()) {
40         WARN_PRINTLN("MQTT未连接, 无法上报结果");
41         return;
42     }
43     // 构建属性JSON (包含完整识别信息)
44     char properties[384];
45     snprintf(properties, sizeof(properties),
46         "\"Status\": \"识别完成\", \"GarbageType\": \"%s\", \"Confidence\": %.2f, \"ImageUrl\"
47         \": \"%s\", \"ObjectName\": \"%s\", \"Timestamp\": \"%s\"",
48         GARBAGE_NAMES[type], confidence, objectUrl, objectName, timestamp);
49     // 构建完整消息 (使用预定义格式)
50     char jsonBuf[512];
51     snprintf(jsonBuf, sizeof(jsonBuf), MQTT_BODY_FORMAT, properties);
52     // 发布消息 (省略)
53 }

```

C. 规则引擎与数据持久化路由：为实现识别记录的零代码自动化存储，我们配置了华为云 IoTDA 的规则引擎。

- **SQL 过滤逻辑：**通过配置 SQL 过滤语句 `SELECT * FROM DEVICE_PROPERTY_REPORT`，并设定过滤条件为 `Status IN ('识别完成')`，精准捕获有效的识别报文。
- **异步转发路由：**规则引擎将匹配的数据实时路由至 OBS 存储桶 `iotda-obs-data` 中的 `Images/records.csv` 文件。转发字段涵盖了物模型中的所有关键维度及事件时间 (`event_time`)，构建了完整的数据链路闭环。

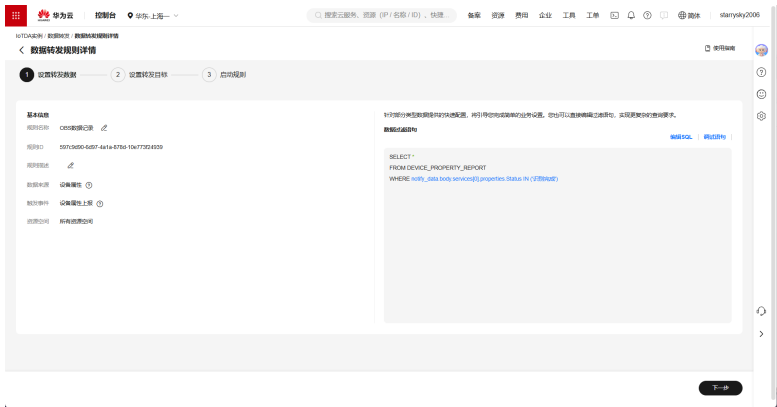


图 6: 数据转发设置

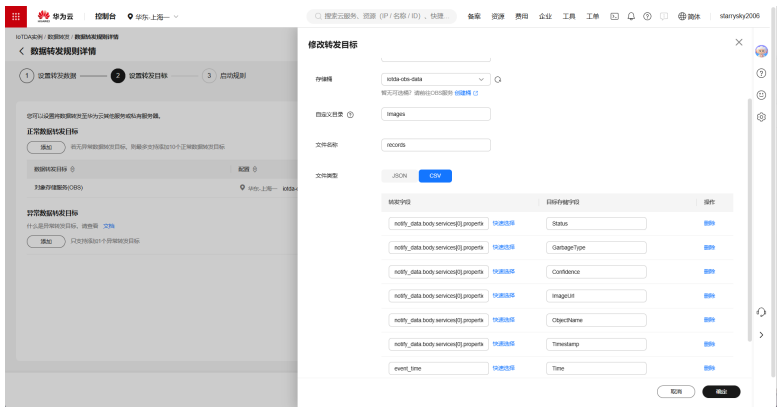


图 7: 数据转发目标

4.5 云端计算

云端计算层主要负责处理来自 ESP32-S3 上传的图像数据，调用大模型进行语义识别，并将结果下发至硬件终端及同步至用户前端。并且通过使用预签名技术实现了安全、低负载的端云协同。

A. 对象存储与安全认证我们在搭建系统时采用 OBS 存储原始采集图像，并应用了安全认证机制。

- **AWS Signature V4 算法实现：**由于嵌入式板端内存有限，系统在 `obs.cpp` 中利用 `mbedtls` 硬件加速器实现了二进制 HMAC 派生密钥计算。上传请求包含了规范化请求头与数据负载哈希，确保了图片在传输过程中的完整性与不可篡改性。我们的算法核心逻辑涉及多级 HMAC-SHA256 哈希派生，如下所示：

Listing 7: 基于 mbedtls 的 SigV4 派生密钥计算逻辑

```

1  uint8_t kDate[32], kRegion[32], kService[32], kSigning[32];
2      String kSecret = "AWS4" + String(OBS_SECRET_KEY);
3      // 第一步: HMAC(Secret Key, Date)
4      hmacSHA256Binary((const uint8_t*)kSecret.c_str(), kSecret.length(), date, kDate)
5          ;
6      // 第二步: HMAC(kDate, Region)
7      hmacSHA256Binary(kDate, 32, OBS_REGION, kRegion);
8      // 第三步: HMAC(kRegion, Service)
9      hmacSHA256Binary(kRegion, 32, "s3", kService);
10     // 第四步: HMAC(kService, "aws4_request")
11     hmacSHA256Binary(kService, 32, "aws4_request", kSigning);
12     // 计算签名 (使用二进制密钥)
13     uint8_t signatureBin[32];
14     hmacSHA256Binary(kSigning, 32, stringToSign, signatureBin);
15     // 将签名转换为十六进制字符串
16     String signature = "";
17     for (int i = 0; i < 32; i++) {
18         if (signatureBin[i] < 16) {
19             signature += "0";
20         }
21         signature += String(signatureBin[i], HEX);
22     }

```

B. 视觉大模型推理在图像识别判别垃圾种类时，我们一开始想要采用 OBS 触发 FunctionGraph 函数调用 ModelArts 已部署的物体检测模型 API 返回垃圾类别，但是训练模型过于麻烦调用开源数据集工作量也较大。于是我们最终选择调用 Qwen3.5-Flash 模型直接进行垃圾分类判别。

- **提示词设计**：为了实现垃圾分类，我们定义了严格的任务描述，明确要求 AI 执行“二分类”任务，并指定以 JSON 格式（包含 type 与 confidence 字段）返回结果。
- **异步请求处理**：我们将响应超时阈值设置为 65 秒，并关闭了模型深度思考模式，以在保证准确度的前提下尽可能缩短端到端的响应时间。

Listing 8: AI 识别结果的结构化解析逻辑

```

1  bool parseAIResponse(const String& response, GarbageType& type, float& confidence) {
2      INFO_PRINTLN("解析AI响应...");
3      // 解析JSON响应 (和之前一样省略...)
4      // 提取AI返回的内容
5      const char* content = doc["choices"][0]["message"]["content"];
6      if (content == NULL) {
7          ERROR_PRINTLN("无法提取AI内容");
8          return false;
9      }
10     INFO_PRINTF("AI内容: %s\n", content);
11
12     // 解析AI返回的JSON内容
13     JsonDocument contentDoc;
14     error = deserializeJson(contentDoc, content);
15     if (error) {
16         ERROR_PRINTLN("AI内容JSON解析失败");
17     }
18 }

```

```

17     // 尝试从文本中提取类型（容错处理）
18     if (strstr(content, "recyclable") != NULL) {
19         type = GARBAGE_RECYCLABLE;
20         confidence = 0.7;
21         INFO_PRINTLN("从容错处理中识别为可回收垃圾");
22         return true;
23     } else if (strstr(content, "other") != NULL) {
24         type = GARBAGE_OTHER;
25         confidence = 0.7;
26         INFO_PRINTLN("从容错处理中识别为其他垃圾");
27         return true;
28     }
29     return false;
30 }
31 // 提取类型和置信度
32 const char* typeStr = contentDoc["type"];
33 confidence = contentDoc["confidence"] | 0.8;
34 if (typeStr == NULL) {
35     ERROR_PRINTLN("无法提取垃圾类型");
36     return false;
37 }
38 // 转换为枚举类型
39 if (strcmp(typeStr, "recyclable") == 0) {
40     type = GARBAGE_RECYCLABLE;
41 } else if (strcmp(typeStr, "other") == 0) {
42     type = GARBAGE_OTHER;
43 } else {
44     ERROR_PRINTF("未知的垃圾类型: %s\n", typeStr);
45     return false;
46 }
47 INFO_PRINTF("识别结果: %s (%.2f%%)\n", GARBAGE_NAMES[type], confidence * 100);
48 return true;
49 }

```

C. 低负载识图链路：相比传统的 Base64 直传方案，我们采用了“图片先存后识”的优化架构。

- **内存消耗优化：**传统方案中，Base64 编码会导致数据体积增加约 33%，易触发 ESP32 的内存溢出。我们通过采用预签名 URL，HTTP 请求体大小降至约 2KB，显著降低了设备端的 RAM 占用。
- **传输可靠性提升：**小体积的 URL 报文在不稳定的 WiFi 环境下具有更高的请求成功率，有效缓解了因网络波动导致的 AI 识别中断问题。

4.6 微信小程序模块

在微信小程序层面，我们采用采用单页面架构设计，承担了指令下发和异步识别结果的检索与可视化，实现了华为云与 OBS 的集成。主要有以下模块：

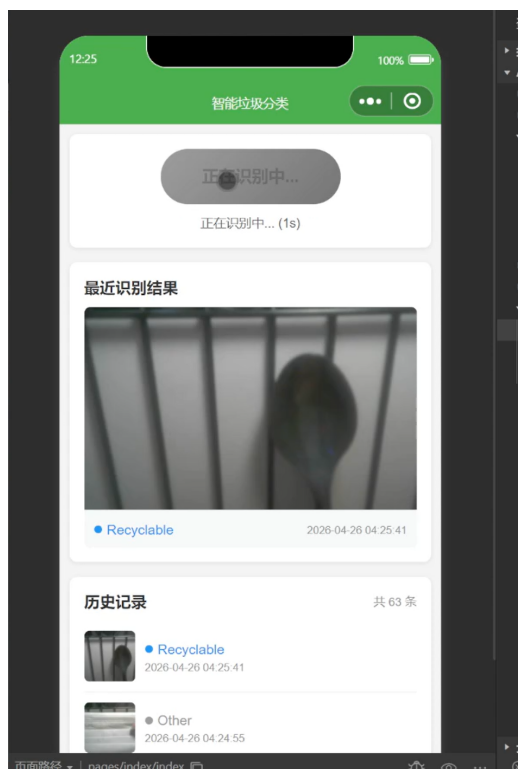


图 8: 微信小程序界面

A. 基于 IAM 认证的指令下发逻辑：小程序通过华为云设备接入服务实现对硬件的远程触发。

- **Token 机制：**通过编写 `getToken()` 函数，系统实现了利用 IAM 账号凭证获取 `X-Auth-Token`，并将其本地缓存以减少认证频次。
- **异步命令执行：**当用户点击“拍照识别”时，小程序向 IoTDA 接口 `POST capture` 命令报文。若 Token 失效，模块具备自动清除缓存并触发重试的容错能力。

B. 异步状态同步与 CSV 轮询机制：考虑到硬件端执行“采集-上传-识别”的全链路过程具有 5-10 秒的延迟，我们在小程序设计中引入了动态时间戳参数，采用了异步状态同步方案。

Listing 9: 小程序异步轮询实现

```

1 startPolling() {
2     var that = this;
3     console.log('[轮询开始] lastRecordCount:', that.data.lastRecordCount);
4     // 每 3 秒轮询一次
5     this._pollTimer = setInterval(function() {
6         var newPollCount = that.data.pollCount + 1;
7         console.log('[轮询] 第', newPollCount, '次查询');
8
9         obs.getRecordCsv()
10            .then(function(csvText) {
11                console.log('[轮询] CSV 内容长度:', csvText.length);
12                var records = obs.parseRecordCsv(csvText);
13                var newCount = records.length;
14                console.log('[轮询] 解析后记录数:', newCount);
15                console.log('[轮询] lastRecordCount:', that.data.lastRecordCount);
16                console.log('[轮询] 是否有新记录:', newCount > that.data.lastRecordCount);
17                // 只更新 pollCount，不更新其他数据，避免频繁刷新

```

```

18     that.setData({ pollCount: newPollCount });
19     if (newCount > that.data.lastRecordCount) {
20         // 发现新记录，停止轮询
21         that.stopPolling();
22         // 更新历史记录
23         var newRecord = records[0]; // 最新的记录 (已 reverse)
24         var latestRecord = withDisplayFields(newRecord);
25
26         console.log('[轮询] 新记录详情:', JSON.stringify(latestRecord, null, 2));
27         var totalPages = Math.ceil(records.length / that.data.pageSize);
28         var currentPage = Math.min(that.data.currentPage, totalPages || 1);
29
30         that.setData({
31             allHistory: records,
32             lastRecordCount: newCount,
33             totalPages: totalPages,
34             currentPage: currentPage,
35             latestRecord: latestRecord,
36             statusMessage: '识别完成: ' + latestRecord.result,
37             statusType: '',
38             isPolling: false
39         });
40         // 加载最新图片
41         that.loadLatestImage(newRecord);
42         that.loadPage(currentPage);
43         wx.showToast({
44             title: '识别完成',
45             icon: 'success'
46         });
47
48         return;
49     } else {
50         // 没有新记录，只更新状态文字，不刷新数据
51         that.setData({
52             statusMessage: '正在识别中... (' + newPollCount + 's)',
53             statusType: ''
54         });
55     }
56
57     // 超时检查 (30 秒)
58     if (newPollCount >= 10) {
59         that.stopPolling();
60         var totalPages = Math.ceil(records.length / that.data.pageSize);
61         var currentPage = Math.min(that.data.currentPage, totalPages || 1);
62
63         // 超时后才更新数据
64         that.setData({
65             isPolling: false,
66             statusMessage: '识别超时，请手动刷新查看',
67             statusType: 'error',
68             allHistory: records,
69             lastRecordCount: newCount,

```

```

70         totalPages: totalPages,
71         currentPage: currentPage,
72         latestRecord: records.length > 0 ? withDisplayFields(records[0]) : {},
73         latestImageUrl: records.length > 0 ? that.data.latestImageUrl : '',
74         latestImageLoaded: records.length === 0,
75         displayHistory: records.length > 0 ? that.data.displayHistory : []
76     });
77     if (records.length > 0) {
78         that.loadLatestImage(records[0]);
79         that.loadPage(currentPage);
80     }
81 }
82 })
83 .catch(function(err) {
84     console.error('[轮询失败]:', err);
85     // 静默失败, 继续轮询, 只更新状态文字
86     that.setData({
87         statusMessage: '正在识别中... (网络异常, 继续尝试)',
88         statusType: 'error'
89     });
90 });
91 }, 3000);
92 },
93 // 停止轮询 (省略)
94 },

```

- **轮询算法:** 下发指令后, 小程序启动一个每 3 秒执行一次的定时器, 请求 OBS 存储桶中的 `records.csv`。为规避微信端的缓存机制, 请求 URL 附带了动态时间戳参数。
- **完成判定:** 小程序通过对比当前读取到的 CSV 总行数与本地记录的 `lastRecordCount` 是否增加, 来判定硬件端是否完成了新的识别任务。一旦发现记录增加, 立即停止轮询并更新 UI 视图。

C. 安全存储访问与图片预览: 为了在保持 OBS 桶私有安全的前提下展示图片, 小程序采用了前端签名预览逻辑。

- **V2 签名实现:** 利用纯 JavaScript 实现了 HMAC-SHA1 算法, 根据 OBS V2 规范生成 `StringToSign`。
- **临时预览 URL:** 系统通过 `getImageUrl()` 为每一张识别图片生成带有有效期的临时访问链接, 确保用户能安全地查看历史识别大图。

D. 异常分级提示机制: 为了在出现故障时提升排错便利度, 我们将常见的云端及硬件故障分为四类进行针对性提示:

- **配置错误:** 包括项目 ID、设备 ID 或服务 ID 设置不一致。
- **认证失败:** IAM 账号或权限配置异常。
- **网络异常:** 域名未配置或无法连接云平台。
- **设备离线:** 硬件未上线或 MQTT 连接中断, 导致命令无法下发。

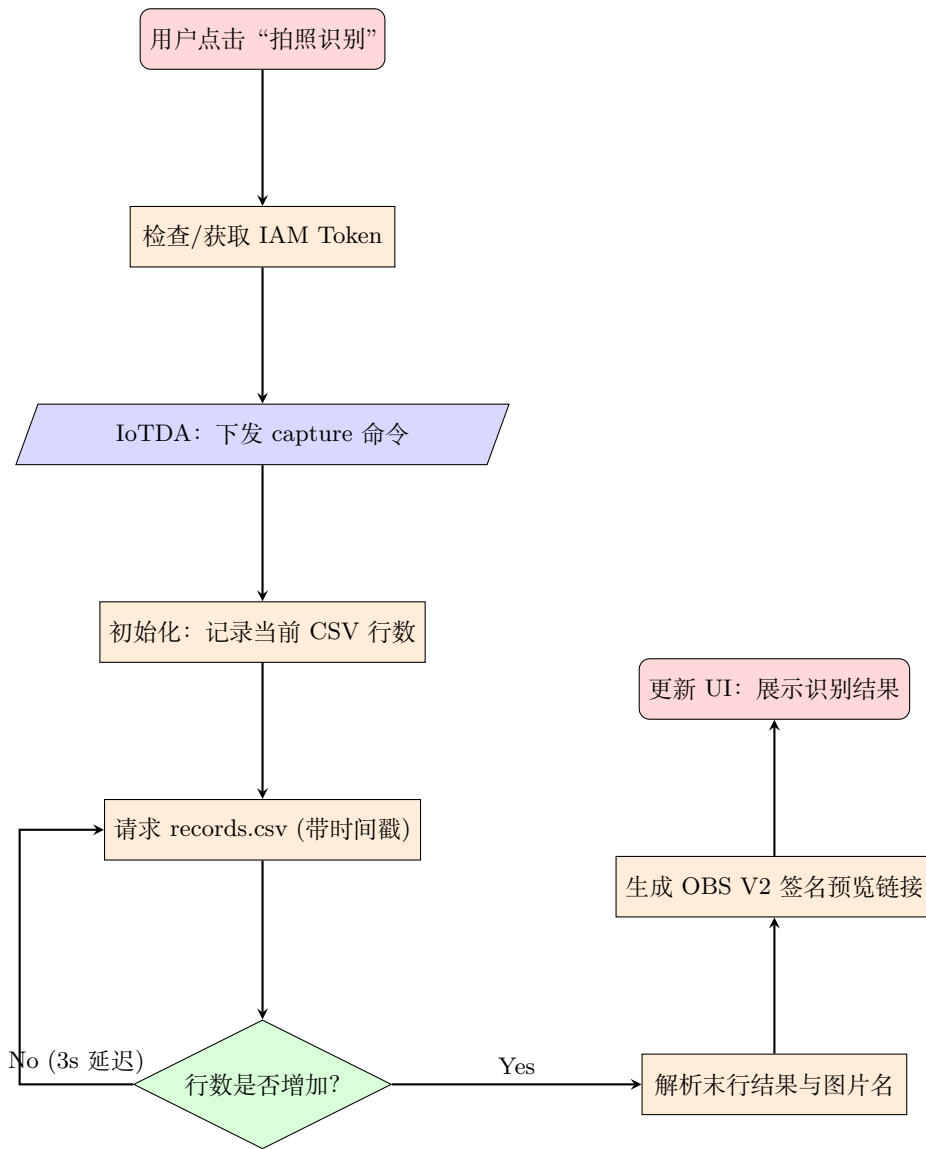


图 9: 微信小程序端异步任务调度与数据可视化流程图

5 实验成果与测试

5.1 测试过程

我们的测试过程采用了分层测试架构，遵循“自底向上”的原则，确保了底层硬件的稳定性和上层逻辑运行。

Layer 1: 硬件基础层 (Hardware Foundation)

此层级旨在独立验证各执行机构和交互组件的正常运行。

- **test_led**: 验证 LED 模块的颜色显示与动态效果。程序依次驱动红、绿、蓝、黄、紫、白 6 种颜色，并测试红色闪烁与蓝色呼吸灯效果和输出信息。通过人工观察 LED 颜色变化和动态效果判断正常运行。
- **test_oled**: 验证本地 UI 界面的显示逻辑。通过模拟数据测试 128×64 像素下的三行文本、进度条动画 (0%–100%) 及识别结果显示。
- **test_servo**: 验证舵机动作与 LED/OLED 的联动反馈。测试舵机在 90° (复位)、45° (其他垃圾) 和 135° (可回收垃圾) 之间的切换。

- **test_servo_simple**: 当 ‘test_servo’ 出现问题时, 使用此最小化测试快速定位是 GPIO 引脚问题、舵机硬件问题还是程序模块问题。直接绑定 GPIO 14 到舵机, 循环使舵机从 0° 逐步转到 100°。

Layer 2: 摄像头层

通过此层级验证摄像头拍摄和存储的正常运行。

- **test_camera**: 实时记录摄像头初始化前后的 Free Heap 与 Free PSRAM 变化, 并进行连续拍照测试, 确保在 640 × 480 分辨率下无内存泄漏。

Layer 3: 网络通信层

通过此层级可以验证设备与华为云 IoTDA 平台之间的长连接与协议稳定性。

- **test_wifi**: 验证网络接入的稳定性与自愈能力。测试 WiFi 扫描、信号强度监测, 并验证基于指数退避策略的自动重试机制。
- **test_iotda**: 验证 MQTT 通信的完整业务逻辑。包括设备影子状态上报、模拟识别结果 (URL 与置信度) 发布, 以及对云端下行 capture 命令的响应。
- **test_iotda_minimal**: MQTT 最小化排障测试。排除项目代码干扰, 通过输出底层的 MQTT 状态码 (如 4 表示凭据错误, 5 表示未授权) 精准定位鉴权故障。

Layer 4: 云服务层

我们通过此层级验证涉及安全鉴权与跨平台 API 调用的端到端业务闭环。

- **test_obs**: 验证安全存储与签名算法。首先通过 NTP 协议同步网络时间, 随后上传测试图片并验证生成的预签名对象 URL 是否可直接访问。
- **test_ai**: 完整模拟 “图像采集 → 安全上传 OBS → 生成预签名 URL → 调用 DashScope API → 解析识别结果” 的闭环。

5.2 实验成果

我们通过对系统进行全链路集成测试, 系统在识别精度、响应时延以及运行稳定性方面均达到了设计预期, 成功实现了从垃圾投放到分类执行的自动化闭环。我们的实际硬件架构如下图:

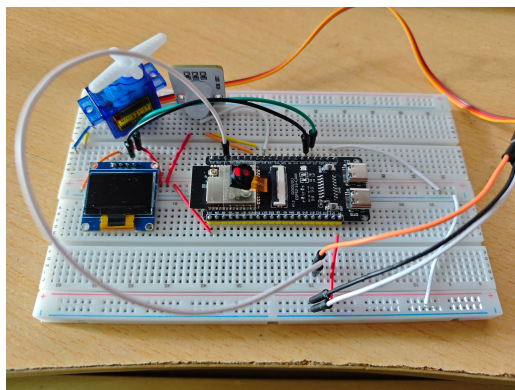


图 10: 产品实体

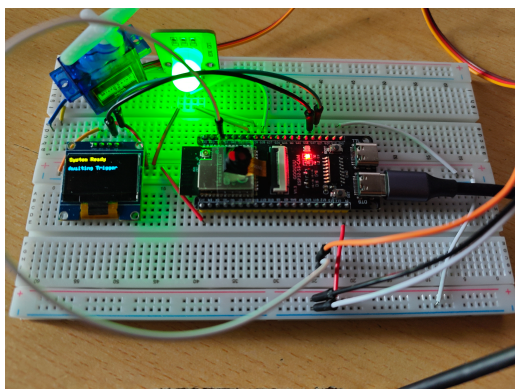


图 11: 产品工作状态

通过以上测试证明了我们的模型具有一定的优越性, 有较快的响应速度、关键节点耗时短、能在资源受限的嵌入式端展现了良好的稳健性并实现了数据流的闭环管理。同时, 视觉大模型的引入实现了对常见垃圾种类的高精度判别。

表 1: 系统性能指标实测结果汇总

测试维度	实测性能指标
识别平均置信度	0.95
端到端响应时延	6.5s - 8.5s
MQTT 连接成功率	100%
舵机角度偏差	$< 1.5^{\circ}$

6 结论

6.1 总结

我们设计并实现了一套基于 ESP32-S3-CAM 与华为云 AIoT 平台的智能垃圾分类识别系统。通过四层架构的深度协同, 成功实现了智能垃圾分类系统设计。

我们的系统设计核心总结如下:

- **架构设计:** 系统采用了“设备端感知-云端智能推理-应用端异步闭环”的架构模式。通过有限状态机实现了非阻塞的硬件调度, 确保了系统在资源受限环境下的稳定性。
- **技术创新:** 针对嵌入式端内存溢出难题, 我们采用了“低负载识图链路”。利用 OBS 预签名 URL 技术代替传统的 Base64 直传方案, 降低了设备端请求负载, 显著提升了图像传输的可靠性。
- **云原生集成:** 充分利用华为云 IoTDA 规则引擎实现了数据流的自动化路由。通过 SQL 过滤机制实现了识别记录的零代码持久化存储, 构建了从物理感知到结构化数据的完整链路。
- **应用体验:** 微信小程序端通过 V2 签名算法与 CSV 异步轮询机制, 实现了安全、流畅的远程控制与结果展示, 端到端平均响应时延稳定在 6.5s-8.5s。

6.2 不足与展望

尽管系统已初步实现了预定功能, 但在实际应用场景中仍存在进一步优化的空间:

- **边缘计算加速**：虽然云端推理精度高，但受限于网络延迟。未来可尝试在 ESP32-S3 上部署轻量化的 TinyML 模型进行初筛，仅将疑似目标上传云端，从而进一步缩短响应时间。
- **能耗管理**：针对移动监测场景，需进一步研究设备在空闲状态下的深度睡眠逻辑（Deep Sleep），以延长电池供电下的续航时间。

综上所述，我们的设计为智能环卫与垃圾分类的自动化处理提供了一种低成本、高可靠、易扩展的参考方案，具有一定的工程实践意义。

7 附录：代码构成

代码源文件另外置于文件夹中。

7.1 ESP32

`main.ino`

功能：系统调度中心。定义了全局状态标志 `captureTriggered` 及有限状态机逻辑。驱动系统在空闲、采集、上传、识别、执行及错误处理状态间流转，并集成了 WDT 以确保系统稳定性。

`camera.cpp`

功能：视觉感知模块。负责 OV3660 传感器的初始化与参数调优。该模块能自动检测硬件 PSRAM，并动态配置双缓冲区与高质量 JPEG 编码参数，同时实现了图像翻转以适配垃圾桶的垂直安装角度。

`obs.cpp`

功能：云端存储安全。在板端实现了 AWS Signature V4 签名算法，通过多级 HMAC-SHA256 哈希派生确保图片上传至 OBS 的安全性。此外，该模块还负责生成带有有效期的预签名 URL，供 AI 推理引擎访问私有存储桶。

`iotda.cpp`

功能：物联网协议通信。负责与华为云 IoTDA 平台建立 MQTT 安全连接，实现属性上报与指令下发监听。通过提取 `request_id` 实现同步命令响应，有效避免了云端接口调用超时。

`ai.cpp`

功能：智能推理接口。封装了对 DashScope 视觉大模型的 API 调用逻辑。通过提示词，强制 AI 返回符合 JSON 格式的识别结果，实现了非结构化图像到结构化数据的转换。

`network.cpp`

功能：主要负责建立与维护设备的基础联网环境，是整个 AIoT 系统能够与华为云及 AI 大模型进行通信的前提。

`servo.cpp & led.cpp & display.cpp`

功能：物理执行与人机交互。分别负责驱动分类舵机精准旋转至预设角度、RGB 状态灯的颜色编码反馈以及 OLED 屏幕的实时文本状态展示。

tikz

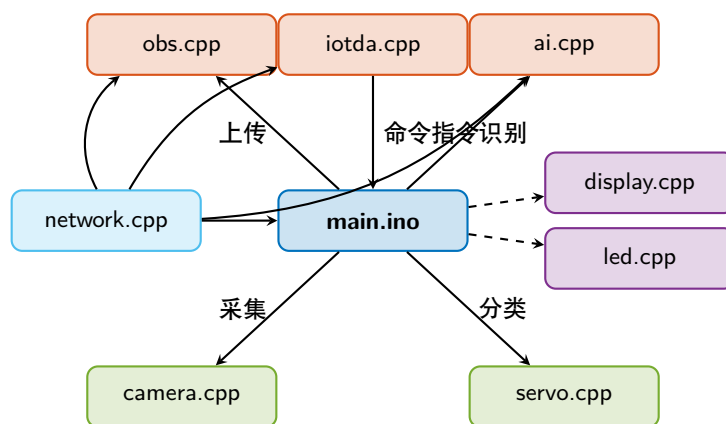


图 12: ESP32-S3-CAM 垃圾分类系统软件模块关系图

7.2 微信小程序

7.2.1 项目根目录配置文件

app.js

功能：全局入口文件。管理小程序的生命周期（如 onLaunch 阶段的本地存储初始化），并在此实现全局登录态的逻辑管理。

app.json

功能：全局配置文件。定义页面路由路径、窗口表现、底部 Tab 栏以及全局网络超时参数。

app.wxss

功能：全局样式文件。定义全域通用的 Flex 布局规范、主题背景颜色及标准字体样式。

project.config.json

功能：项目配置文件。包含微信 AppID、编译配置、库版本信息，确保团队开发环境的高度一致性。

sitemap.json

功能：搜索索引配置。规定微信搜索爬虫对小程序内页面的索引权限。

7.2.2 核心业务页面

index.js

功能：核心业务逻辑。实现“一键拍照”指令下发、通过 IoTDA API 进行设备影子状态轮询、以及识别结果记录的解析显示。

index.wxml

功能：页面模板。构建包含状态指示灯、实时监测卡片、历史识别列表及图片预览窗口的响应式布局。

index.wxss

功能：页面样式。

index.json

功能：页面级配置。设置页面特有的窗口属性。

辅助服务与工具

huawei-iot.js

功能：华为云连接服务。封装了 IAM 鉴权获取 Token 的请求逻辑，并实现了通过 MQTToverHTTPS 协议下发设备指令的底层接口。

obs-service.js

功能：对象存储服务。在 JS 端独立实现 OBS V2 签名算法，负责生成带有效期的图片访问 URL 及解析云端 records.csv。

util.js

功能：基础工具类。提供通用的日期时间格式化、字符串处理及数据校验辅助函数。

7.3 调试与验证脚本

test_sha1_debug.js

功能：前端算法调试工具。在小程序沙箱环境内模拟 SHA-1 计算，确保 JS 端签名逻辑与嵌入式端、云端算法的一致性。

verify_obs_access.py

功能：权限验证脚本。用于测试 OBS 桶及对象的 ACL 策略，确保生成的预签名链接在安全隔离下可被前端正常访问。

check_csv_content.py & verify_sha1_rounds.py

功能：数据与算法一致性校验。分别用于检查 OBS 云端记录内容的准确性，以及逐轮比对哈希计算过程以修复逻辑缺陷。

upload_test_data.py & verify_obs_images.py

功能：环境初始化与维护。用于批量上传模拟测试集及验证云端存储对象的命名路径规范。